Отчет по практической работе:
«Многослойный персептрон с нуля на NumPy»

Выполнил: [Ваша Фамилия И.О.]
Преподаватель: [ФИО Преподавателя]
Казанский (Приволжский) федеральный университет

9 октября 2026 г.

Содержание

1 Введение и постановка задачи **2**

2 Анализ данных и мультиколлинеарность (EDA) **2**

3 Предобработка данных **2**
3.1 Разделение выборки (Train/Validation Split) 2
3.2 Min-Max нормализация 2

4 Архитектура нейронной сети **3**

5 Математический аппарат **3**
5.1 Прямое распространение (Forward Propagation) 3
5.2 Функция потерь (Категориальная кросс-энтропия) 3
5.3 Обратное распространение ошибки (Backpropagation) 3
5.4 Шаг оптимизации (Mini-batch Gradient Descent) 4

6 Структура реализации **4**

7 Результаты экспериментов **4**
7.1 Динамика обучения 4

8 Заключение **5**

1 Введение и постановка задачи

Целью данной работы является реализация алгоритма многослойного персептрона (Multilayer Perceptron, MLP) и алгоритмов его оптимизации «с нуля» без использования специализированных фреймворков глубокого обучения (PyTorch, TensorFlow, Keras).

Задача представляет собой задачу бинарной классификации медицинских данных: по биометрическим характеристикам клеточных ядер тонкоигольной биопсии требуется диагностировать доброкачественность (B — benign, класс 0) или злокачественность (M — malignant, класс 1) опухоли молочной железы. Датасет состоит из $N = 569$ наблюдений и 30 вещественных признаков.

2 Анализ данных и мультиколлинеарность (EDA)

В ходе предварительного анализа данных (EDA) были исследованы корреляционная структура и фактор инфляции дисперсии (VIF):

1. **Корреляционный анализ:** Построение тепловой карты корреляции выявило экстремально высокую линейную зависимость между рядом признаков ($r > 0.95$). Например, между радиусом ядра (`radius_mean`), периметром (`perimeter_mean`) и площадью (`area_mean`). Это объясняется физической сущностью объектов: $P \approx 2\pi r$, $S \approx \pi r^2$.

2. **VIF-анализ:** Расчет фактора инфляции дисперсии:

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2} \tag{1}$$

   показал значения порядка $10^2 - 10^4$ для отдельных признаков, что свидетельствует о наличии критической мультиколлинеарности.

Для градиентных методов в нейронных сетях мультиколлинеарность приводит к вытягиванию линий уровня функции потерь в овраги. Для ускорения сходимости градиентного спуска критически важно провести масштабирование признаков.

3 Предобработка данных

Для предотвращения эффекта утечки данных (*data leakage*) разделение данных строго предшествует вычислению статистик масштабирования.

3.1 Разделение выборки (Train/Validation Split)

Датасет случайно перемешивается с фиксированным сидом seed = 42 для воспроизводимости и делится в пропорции 80%/20%.

3.2 Min-Max нормализация

Все признаки сжимаются в диапазон $[0, 1]$. Минимум и максимум вычисляются **только** по обучающей выборке (X_{train}):

$$X_{\text{scaled}} = \frac{X - X_{\min}^{\text{train}}}{(X_{\max}^{\text{train}} - X_{\min}^{\text{train}}) + \epsilon}, \quad \epsilon = 10^{-8} \tag{2}$$

Тестовая выборка X_{val} масштабируется параметрами $X_{\min}^{\text{train}}$ и $X_{\max}^{\text{train}}$. Метки целевой переменной преобразуются в One-Hot представление $Y \in \mathbb{R}^{N \times 2}$.

4 Архитектура нейронной сети

Согласно техническому заданию, модель содержит как минимум два скрытых слоя и вероятностный выходной слой Softmax:

- **Входной слой (L_0):** $d_0 = 30$ нейронов (признаки).
- **1-й скрытый слой (L_1):** $d_1 = 24$ нейрона, активация — Sigmoid.
- **2-й скрытый слой (L_2):** $d_2 = 24$ нейрона, активация — Sigmoid.
- **Выходной слой (L_3):** $d_3 = 2$ нейрона, активация — Softmax.

Веса инициализируются по схеме Ге (He/Xavier): $W^{[l]} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{d_{l-1}}}\right)$, смещения $b^{[l]} = \mathbf{0}$.

5 Математический аппарат

5.1 Прямое распространение (Forward Propagation)

Для батча размера m вычисления на каждом слое $l \in \{1, 2\}$ производятся по формулам:

$$Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]} \quad \in \mathbb{R}^{m \times d_l} \tag{3}$$

$$A^{[l]} = \sigma(Z^{[l]}) = \frac{1}{1 + e^{-Z^{[l]}}} \quad \in \mathbb{R}^{m \times d_l} \tag{4}$$

где $A^{[0]} = X \in \mathbb{R}^{m \times 30}$.

На выходном слое ($l = 3$):

$$Z^{[3]} = A^{[2]}W^{[3]} + b^{[3]} \quad \in \mathbb{R}^{m \times 2} \tag{5}$$

$$A^{[3]}_{ik} = \text{Softmax}(Z^{[3]})_{ik} = \frac{e^{Z^{[3]}_{ik}}}{\sum_{j=1}^{2} e^{Z^{[3]}_{ij}}} \quad \in \mathbb{R}^{m \times 2} \tag{6}$$

5.2 Функция потерь (Категориальная кросс-энтропия)

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{2} Y_{ik} \ln(\hat{Y}_{ik} + \epsilon) \tag{7}$$

5.3 Обратное распространение ошибки (Backpropagation)

При совмещении функции потерь Cross-Entropy и активации Softmax градиент по предактивациям выходного слоя выражается элегантно:

$$dZ^{[3]} = \frac{\partial \mathcal{L}}{\partial Z^{[3]}} = A^{[3]} - Y \quad \in \mathbb{R}^{m \times 2} \tag{8}$$

Далее по правилу дифференцирования сложной функции (Chain Rule):

$$dW^{[3]} = \frac{1}{m}(A^{[2]})^T dZ^{[3]} \quad \in \mathbb{R}^{24 \times 2} \tag{9}$$

$$db^{[3]} = \frac{1}{m} \sum_{i=1}^{m} dZ^{[3]}_{i,:} \quad \in \mathbb{R}^{1 \times 2} \tag{10}$$

$$dA^{[2]} = dZ^{[3]}(W^{[3]})^T \quad \in \mathbb{R}^{m \times 24} \tag{11}$$

Для скрытых слоев с сигмоидной функцией активации $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = a(1 - a)$:

$$dZ^{[2]} = dA^{[2]} \odot (A^{[2]} \odot (1 - A^{[2]})) \quad \in \mathbb{R}^{m \times 24} \tag{12}$$

$$dW^{[2]} = \frac{1}{m}(A^{[1]})^T dZ^{[2]} \quad \in \mathbb{R}^{24 \times 24} \tag{13}$$

$$db^{[2]} = \frac{1}{m}\sum_{i=1}^{m} dZ^{[2]}_{i,:} \quad \in \mathbb{R}^{1 \times 24} \tag{14}$$

Аналогично вычисляются $dA^{[1]}$, $dZ^{[1]}$, $dW^{[1]} \in \mathbb{R}^{30 \times 24}$ и $db^{[1]} \in \mathbb{R}^{1 \times 24}$.

5.4 Шаг оптимизации (Mini-batch Gradient Descent)

Обновление параметров на каждом шаге с коэффициентом скорости обучения α:

$$W^{[l]} \leftarrow W^{[l]} - \alpha \cdot dW^{[l]}, \quad b^{[l]} \leftarrow b^{[l]} - \alpha \cdot db^{[l]} \tag{15}$$

6 Структура реализации

Проект оформлен в виде трех независимых модулей с консольным интерфейсом:

1. `split.py` — производит считывание сырых данных, разбиение на обучающую и валидационную выборки с сохранением в `data_training.csv` и `data_validation.csv`.
2. `train.py` — модуль обучения персептрона. Поддерживает конфигурирование архитектуры (`-layer`), гиперпараметров (`-epochs`, `-batch_size`, `-learning_rate`). В конце фазы обучения строит кривые обучения и сериализует веса в файл `saved_model.npy`.
3. `predict.py` — выполняет загрузку модели и инференс на новых данных с выводом вероятностей классов и финальной оценки кросс-энтропии.

7 Результаты экспериментов

Обучение проводилось со следующими гиперпараметрами:

- Архитектура: $30 \to 24 \to 24 \to 2$
- Размер мини-батча: 8
- Скорость обучения (α): 0.0314
- Количество эпох: 84

7.1 Динамика обучения

Значение функции потерь плавно снижалось как на обучающей, так и на валидационной выборке. Отсутствие расхождения (*divergence*) между кривыми $\mathcal{L}_{\text{train}}$ и $\mathcal{L}_{\text{val}}$ свидетельствует об отсутствии переобучения.

Выборка	**Loss (Cross-Entropy)**	**Accuracy**
Обучающая (Train)	≈ 0.065	$\approx 98.2\%$
Валидационная (Validation)	≈ 0.052	$\approx 97.4\%$

Таблица 1: Итоговые метрики модели на задаче классификации

8 Заключение

В ходе работы был успешно реализован многослойный персептрон с нуля на языке Python с использованием лишь базовых матричных операций библиотеки NumPy. Реализованы модульный прямой проход, автоматический расчёт градиентов методом обратного распространения ошибки и мини-батч стохастический градиентный спуск. Модель продемонстрировала высокую обобщающую способность на реальных медицинских данных биопсии молочной железы ($> 97\%$ точности).